optimization

Paolo Colussi

2025-11-04

Appendix – Derivation of the Small-System Formulas

One of the things that makes the GMERT algorithm computationally expensive is the need to invert the matrix $\hat{V}_j$ of size $nj \times n_j$ at each iteration for each group j for the updates of the random effects $\hat{b}_j$, the residual variance $\hat{\sigma}^2$, and the random-effects covariance $\hat{D}$. What can make this feasible is that the number of random effects q is typically much smaller than the number of observations per group n_j. Using the Woodbury matrix identity, we can rewrite these updates in terms of inverses of $q \times q$ matrices instead of $n_j \times n_j$ matrices, which is computationally much cheaper. This appendix derives these small-system formulas.

1. Original GMERT Updates

First, we restate the relevant definitions from the GMERT paper. At iteration m, the paper defines the random-effects, residual, and covariance updates as:

$$
\begin{aligned}
\hat{b}_j^{(m)} &= \hat{D}_{(m-1)}(W_j^{1/2(M)} Z_j)^\top \hat{V}_j^{-1(m-1)} W_j^{1/2(M)} \big(\tilde{y}_j^{(M)} - \hat{f}_{(m)}(X_j)\big), \\
\hat{V}_j^{(m-1)} &= W_j^{1/2(M)} Z_j \hat{D}_{(m-1)} Z_j^\top W_j^{1/2(M)} + \hat{\sigma}^2_{(m-1)} I_{n_j}, \\
\hat{\sigma}^2_{(m)} &= \frac{1}{N} \sum_{j=1}^{n} \Big\{ \hat{\epsilon}_i^\top \hat{\epsilon}_j + \hat{\sigma}^2_{(m-1)} \big[n_j - \hat{\sigma}^2_{(m-1)} \operatorname{tr}(\hat{V}_j^{-1(m-1)}) \big] \Big\}, \\
\hat{D}_{(m)} &= \frac{1}{n} \sum_{j=1}^{n} \Big\{ \hat{b}_j^{(m)} \hat{b}_j^{(m)\top} + \big[\hat{D}_{(m-1)} - \hat{D}_{(m-1)} (W_j^{1/2} Z_j)^\top \hat{V}_j^{-1(m-1)} W_j^{1/2} Z_j \hat{D}_{(m-1)} \big] \Big\},
\end{aligned}
$$

with

$$
\hat{\epsilon}_j^{(m)} = W_j^{1/2} \big(\tilde{y}_j - \hat{f}_j - Z_j \hat{b}_j^{(m)} \big).
$$

All inverses are over the $n_j \times n_j$ matrices $\hat{V}_j$, which dominates computational cost.

---

2. Woodbury Identity

To derive small-system expressions, we use the Woodbury matrix identity: For matrices $A \in \mathbb{R}^{n_i \times n_j}$, $U \in \mathbb{R}^{n_j \times q}$, and $C \in \mathbb{R}^{q \times q}$,

$$
(A + UCU^\top)^{-1} = A^{-1} - A^{-1} U \, (C^{-1} + U^\top A^{-1} U)^{-1} U^\top A^{-1}.
$$

Set $A = \sigma^2 I_{n_j}$, $U = W_j^{1/2} Z_j$, $C = D$. Then

$$\boxed{V_j^{-1} = \sigma^{-2} I_{n_j} - \sigma^{-4} W_j^{1/2} Z_j \big(D^{-1} + \sigma^{-2} Z_j^\top W_j Z_j\big)^{-1} Z_j^\top W_j^{1/2}.} \tag{1}$$

Define

$$A_j = D^{-1} + \sigma^{-2} Z_j^\top W_j Z_j \quad (q \times q \text{ matrix}).$$

In R we can implement A_j^{-1} as follows:

```
# Ajnv_fun: precomputes A_j^{-1} (and optionally Z_j^T W_j Z_j) for all clusters
Ajnv_fun <- function(G,         # G      : number of clusters
                     Z,         # Z      : N x q random-effects design
                     w,         # W      : list of weigths
                     D,         # D      : q x q covariance of random effects (current)
                     sigma2,    # sigma2 : residual variance (current)
                     idx       # idx    : list length G with row indices for each cluster
){
  Ajnv_list <- vector("list", G)

  Djnv <- solve(D)                                  # D^{-1} (q x q) once

  for (g in seq_len(G)) {
    Zj  <- Z[idx[[g]], , drop = FALSE]              # n_j x q
    Wj  <- diag(w[idx[[g]]])
    ZtWZ <- t(Zj) %*% Wj %*% Zj                      # Z_j^T W_j Z_j (q x q)

    A <- Djnv + as.numeric((1 / sigma2)) * ZtWZ                 # A_j
    Ajnv <- solve(A)                               # A_j^{-1}

    Ajnv_list[[g]] <- Ajnv
  }

  Ajnv_list
}
```

We calculate directly A_i^{-1} for each group i since it is needed in all three updates, and we don't have to solve the same system three times.

---

3. Small-System Expression for b_i

Substituting (1) into the paper's definition of $\hat{b}_i$ and simplifying with $W_i^{1/2} Z_i = U_i$ gives

$$\begin{aligned}\hat{b}_i &= D(W_i^{1/2} Z_i)^\top \big[\sigma^{-2} I - \sigma^{-4} W_i^{1/2} Z_i A_i^{-1} Z_i^\top W_i^{1/2}\big] W_i^{1/2} (\tilde{y}_i - \hat{f}_i) \\ &= \big(D^{-1} + \sigma^{-2} Z_i^\top W_i Z_i\big)^{-1} \big(\sigma^{-2} Z_i^\top W_i (\tilde{y}_i - \hat{f}_i)\big).\end{aligned}$$

Hence,

$$\boxed{\hat{b}_i = A_i^{-1} \sigma^{-2} Z_i^\top W_i (\tilde{y}_i - \hat{f}_i), \qquad A_i = D^{-1} + \sigma^{-2} Z_i^\top W_i Z_i.} \tag{2}$$

which is algebraically identical to the paper's expression but only requires solving a $q \times q$ system instead of an $n_i \times n_i$ system.

In R

```
# b_fun_Ainv: updates random effects b_i using precomputed A_i^{-1}
b_fun_small <- function(G,         # G      : number of clusters
                        Z,         # Z      : N x q random-effects design
                        w,         # W      : list of weigths
                        idx,       # idx    : list length G with row indices for each cluster
                        y_t,       # y_t    : pseudo-response vector (length N)
                        fhat,      # fhat   : fitted fixed-part prediction (length N)
                        Ajnv,      # Ainv   : list length G; each Ainv[[g]] = A_i^{-1} (q x q)
                        sigma2     # sigma2 : residual variance (current)
){
  q <- ncol(Z)
  b <- matrix(0, G, q)

  for (g in seq_len(G)) {
    Zi   <- Z[idx[[g]], , drop = FALSE]           # n_i x q
    Wi   <- diag(w[idx[[g]]])                     # weights vector (length n_i)
    resid<- y_t[idx[[g]]] - fhat[idx[[g]]]        # (y_t - fhat) on cluster i

    rhs  <- as.numeric((1 / sigma2)) * t(Zi) %*% (Wi %*% resid)  # right-hand side
    b[g, ] <- as.vector(Ainv[[g]] %*% rhs)           # b_i = A_i^{-1} * rhs
  }

  b
}
```

---

4. Trace of V_i^{-1}

Using (1) and the cyclic property of the trace, $\mathrm{tr}(ABC) = \mathrm{tr}(BCA) = \mathrm{tr}(CAB)$ (with $A = W_i^{1/2} Z_i$, $B = A_i^{-1}$, $C = Z_i^\top W_i^{1/2}$), we have

$$\mathrm{tr}(V_i^{-1}) = \mathrm{tr}\left(\sigma^{-2} I_{n_i}\right) - \mathrm{tr}\left(\sigma^{-4} W_i^{1/2} Z_i A_i^{-1} Z_i^\top W_i^{1/2}\right) = \frac{n_i}{\sigma^2} - \frac{1}{\sigma^4}\,\mathrm{tr}(A_i^{-1} Z_i^\top W_i Z_i). \tag{3}$$

Thus, the expensive $n_i \times n_i$ trace can be replaced by a $q \times q$ trace.

---

5. Small-System Expression for σ^2

Plug (3) into the paper's σ^2 update:

$$\begin{aligned}
\hat{\sigma}^2 &= \frac{1}{N}\sum_i \Big[\hat{\epsilon}_i^\top \hat{\epsilon}_i + \sigma^2\big(n_i - \sigma^2\ \mathrm{tr}(V_i^{-1})\big)\Big] \\
&= \frac{1}{N}\sum_i \Big[\hat{\epsilon}_i^\top \hat{\epsilon}_i + \sigma^2\big(n_i - \sigma^2\big(\tfrac{n_i}{\sigma^2} - \tfrac{1}{\sigma^4}\,\mathrm{tr}(A_i^{-1}Z_i^\top W_i Z_i)\big)\big)\Big] \\
&= \frac{1}{N}\sum_i \Big[\hat{\epsilon}_i^\top \hat{\epsilon}_i + \mathrm{tr}(A_i^{-1}Z_i^\top W_i Z_i)\Big].
\end{aligned}$$

Hence the compact small-system form:

$$\boxed{\hat{\sigma}^2 = \frac{1}{N}\sum_i \Big[\hat{\epsilon}_i^\top \hat{\epsilon}_i + \mathrm{tr}(A_i^{-1}Z_i^\top W_i Z_i)\Big].} \tag{4}$$

In R

```
# sigma_fun_Ainv: updates residual variance sigma2 using precomputed A_i^{-1}
sigma_fun_small <- function(N,        # N      : total number of rows
                            G,        # G      : number of clusters
                            idx,      # idx    : list length G with row indices for each cluster
                            Z,        # Z      : N x q random-effects design
                            w,        # W      : list of weigths
                            y_t,      # y_t    : pseudo-response vector
                            fhat,     # fhat   : fitted fixed-part prediction
                            b,        # b      : matrix G x q with current random effects
                            Ainv,     # Ainv   : list length G; each Ainv[[g]] = A_i^{-1}
                            sigma2   # sigma2 : residual variance (current)
){
  rss_total <- 0

  for (g in seq_len(G)) {
    Zi  <- Z[idx[[g]], , drop = FALSE]             # n_i x q
    wi  <- diag(w[idx[[g]]])                             # weights matrix
    ni  <- length(wi)

    # residuals eps_i
    eps_i <- diag(sqrt(w[idx[[g]]])) %*% (y_t[idx[[g]]] - fhat[idx[[g]]] - as.vector(Zi %*% b[g, ]))

    # tr(V_i^{-1})
    trVi_inv <- (ni / sigma2) - as.numeric((1 / sigma2^2)) * sum(diag(Ainv[[g]] %*% t(Zi) %*% wi %*% Zi

    # accumulate rss
    rss_total <- rss_total + t(eps_i) %*% eps_i + sigma2 * (ni - sigma2 * trVi_inv)
  }

  rss_total / N
}
```

6. Small-System Expression for D

For what concerns the matrix D, the procedure is slightly more complex. we start from the original update: At outer iteration m, with $U_i \equiv W_i^{1/2} Z_i$ the update is

$$\hat{D}_{(m)} \;=\; \frac{1}{n}\sum_{i=1}^{n}\Big[\hat{b}_i^{(m)}\hat{b}_i^{(m)\top} \;+\; \big(\hat{D}_{(m-1)} - \hat{D}_{(m-1)}U_i^\top V_i^{-1}U_i\,\hat{D}_{(m-1)}\big)\Big]. \tag{D.1}$$

As always we want to avoid the large $n_i \times n_i$ inverse V_i^{-1}, so we aim to replace it by a $q \times q$ inverse. Recall the definitiona

$$A_i \;\equiv\; \hat{D}_{(m-1)}^{-1} \;+\; \hat{\sigma}_{(m-1)}^{-2}\, Z_i^\top W_i Z_i \quad (q \times q). \tag{D.2}$$

and

$$V_i^{-1} = \sigma^{-2} I - \sigma^{-4} U_i A^{-1} U_i^\top$$

We will show

$$\boxed{\hat{D}_{(m-1)} - \hat{D}_{(m-1)}U_i^\top V_i^{-1}U_i\,\hat{D}_{(m-1)} \;=\; A_i^{-1}.} \tag{D.3}$$

Plugging (D.3) into (D.1) yields the optimized update

$$\boxed{\hat{D}_{(m)} \;=\; \frac{1}{n}\sum_{i=1}^{n}\Big[\hat{b}_i^{(m)}\hat{b}_i^{(m)\top} \;+\; A_i^{-1}\Big].} \tag{D.4}$$

Consider the bracket in (D.1), suppressing the index i for readability:

$$C \;\equiv\; D - D\,U^\top V^{-1} U\,D. \tag{D.5}$$

Substitute V^{-1} and expand:

$$\begin{aligned} C &= D - D\,U^\top\big(\sigma^{-2}I - \sigma^{-4}UA^{-1}U^\top\big)UD \\ &= D \;-\; \sigma^{-2}D\,U^\top U\,D \;+\; \sigma^{-4}D\,U^\top U\,A^{-1}\,U^\top U\,D. \end{aligned} \tag{D.6}$$

With $B = \sigma^{-2}U^\top U = \sigma^2 Z_i^\top W_i Z_i$, becomes

$$C \;=\; D \;-\; DBD \;+\; DBA^{-1}BD. \tag{D.7}$$

and A_i becomes:

$$A_i \;=\; D_i^{-1} + B_i.$$

We now prove

$$D \;-\; DBD \;+\; DBA^{-1}BD \;=\; A^{-1}, \tag{D.8}$$

Let $X = D - DBD + DBA^{-1}BD$. We show $AX = I$.

Compute:

$$AD = (D^{-1} + B)D = I + BD, \qquad ADBD = (I + BD)BD = BD + BDBD,$$
$$ADBA^{-1}BD = (I + BD)BA^{-1}BD = BA^{-1}BD + BDBA^{-1}BD.$$

Hence

$$\begin{aligned} AX &= AD - ADBD + ADBA^{-1}BD \\ &= (I + BD) - (BD + BDBD) + (BA^{-1}BD + BDBA^{-1}BD) \\ &= I - BDBD + BA^{-1}BD + BDBA^{-1}BD. \end{aligned} \tag{D.9}$$

From $A^{-1}(D^{-1}+B)=I$, multiply on the right by D to get

$$A^{-1}BD = D - A^{-1}. \tag{D.10}$$

Left-multiply (D.10) by B and by DB, respectively:

$$BA^{-1}BD = BD - BA^{-1}, \tag{D.11}$$

$$DBA^{-1}BD = DBD - DBA^{-1}. \tag{D.12}$$

Now left-multiply (D.12) by B:

$$BDBA^{-1}BD = BDBD - BDBA^{-1}. \tag{D.13}$$

Substitute (D.11) and (D.13) into (D.9):

$$AX = I - BDBD + (BD - BA^{-1}) + (BDBD - BDBA^{-1}) = I + BD - \big(BA^{-1} + BDBA^{-1}\big).$$

Factor the last bracket:

$$BA^{-1} + BDBA^{-1} = B(I+DB)A^{-1}.$$

But $I+DB = D(D^{-1}+B) = DA$, hence $B(I+DB)A^{-1} = BDAA^{-1} = BD$. Therefore $AX = I+BD-BD = I$, proving $X = A^{-1}$.

Applying (D.8) to (D.7) shows $C = A^{-1}$, which is (D.3).

Combining (D.3) with the original update (D.1) yields

$$\boxed{\hat{D}_{(m)} \;=\; \frac{1}{n}\sum_{i=1}^{n}\Big[\hat{b}_i^{(m)}\hat{b}_i^{(m)\top} \;+\; A_i^{-1}\Big], \qquad A_i \;=\; \hat{D}_{(m-1)}^{-1} + \hat{\sigma}_{(m-1)}^{-2}\, Z_i^\top W_i Z_i,}$$

In R

```
# D_fun_Ainv: updates random-effects covariance D using precomputed A_i^{-1}
D_fun_small <- function(G,         # G      : number of clusters
                        b,         # b      : matrix G x q with current random effects
                        Ainv       # Ainv   : list length G; each Ainv[[g]] = A_i^{-1}
){
  q <- ncol(b)
  D_new <- matrix(0, q, q)

  for (g in seq_len(G)) {
    D_new <- D_new + tcrossprod(b[g, ]) + Ainv[[g]]   # b_i b_i^T + A_i^{-1}
  }

  D_new / G
}
```

Summary of Small-System Formulas

All three updates can be computed using only $q \times q$ matrix inverses, this reducing computational cost from $O(n_i^3)$ to $O(q^3)$ per group i.

We can now implement on R a new version of the GMERT updates using these small-system formulas. It is basically the same as the version in the gmert_algo.Rmd, except that we replace the b_fun, sigma_fun, and D_fun functions with the small-system versions derived here, and we add a call to Ainv_fun to precompute the A_i^{-1} matrices at each iteration. So we first of all have to call the function to compute the GLL.

```r
fit_gmert_small    <- function(df,          # df: data.frame with columns
                                            #   id   : cluster identifier (factor or integer)
                                            #   y    : numeric response
                                            #   x1,x2,x3 : numeric covariates
                                            #   leaf : true generating leaf (optional, for diagnostics)
                          max_iter_inn = 1000,  # maximum number of EM iterations (inner loop)
                          max_iter_out = 1000,  # maximum number of PQL iterations (outer loop)
                          tol = 1e-6,           # convergence tolerance for both loops (Aitken or relati
                          cp = 0.0,             # rpart complexity parameter (pruning threshold)
                          minsplit = 50,        # minimum number of obs required to attempt a split
                          minbucket = 20,       # minimum number of obs in any terminal node
                          maxdepth = 5,         # maximum tree depth
                          xval = 10             # number of cross-validation folds in rpart
                          ) {

   # --- Start timer ---
  time_start <- proc.time()              # records user, system, elapsed time

  # --- Basic setup ---
  N <- nrow(df)                                # total number of observations
  G <- length(unique(df$id))                   # number of clusters
  idx_by_cluster <- split(seq_len(N), df$id)   # list: row indices grouped by cluster

  y <- df$y                                    # response vector
  Z <- cbind(1, df$x1)                         # random-effects design: intercept + slope on x1
  q <- ncol(Z)                                 # number of random effects (q = 2)

  # --- Initialization (Step 0) ---
  M <- 0L                                      # outer-loop counter
  mu <- ifelse(y == 1, 0.75, 0.25)             # initial conditional means
  y_t <- log(mu / (1 - mu)) + (y - mu) / (mu * (1 - mu))  # initial pseudo-response (PQL linearization)
  w <- mu * (1 - mu)                           # initial working weights
  sigma2 <- 1                                  # initial residual variance
  D <- diag(2)                                 # initial random-effects covariance (identity)
  b <- matrix(0, G, q)                         # initialize cluster random effects
  gll <- c(0, 0)                               # GLL storage (2 slots for Aitken acceleration)
  eta_old <- rep(0, N)                         # previous eta for outer-loop convergence check
  converged_in <- c()                          # inner-loop convergence flags (per outer iteration)
  converged_out <- FALSE                       # outer-loop convergence flag

  # --- Outer loop (PQL updates) ---
  repeat {
    m = 0                                      # reset inner-loop counter

    # --- Inner loop (EM-like iterations) ---
    repeat {
      m <- m + 1L

      # (1.i) Partial E-step: compute adjusted pseudo-response y_tilde* = y_tilde - Z b
      zb <- numeric(N)                         # cluster-specific random contributions
      for (g in seq_len(G)) {
        idx <- idx_by_cluster[[g]]             # indices for cluster g
        zb[idx] <- Z[idx, , drop = FALSE] %*% b[g, ]  # Z_i b_i
```

```r
    }
    y_star <- y_t - zb                        # adjusted response for tree fit

    # (1.ii) M-step: fit regression tree for fixed effects f(X)
    ctrl <- rpart.control(cp = cp, minsplit = minsplit, xval = xval,
                          minbucket = minbucket, maxdepth = maxdepth)
    Xdf <- data.frame(x1 = df$x1, x2 = df$x2, x3 = df$x3)
    tree <- rpart(y_star ~ x1 + x2 + x3,
                  data = cbind(y_star = y_star, Xdf),
                  weights = w, method = "anova", control = ctrl)
    fhat <- as.numeric(predict(tree, newdata = Xdf))

    # (1.iii) Update random effects b_i
    Ainv <- Ajnv_fun(Z = Z, D = D, sigma2 = sigma2, G = G, idx = idx_by_cluster, w = w)  # build V_i
    b <- b_fun_small(G = G, Z = Z, idx = idx_by_cluster,
               y_t = y_t, fhat = fhat, w = w, sigma2 = sigma2, Ainv = Ainv)

    # (2.i) Update sigma^2 (residual variance)
    sigma2 <- sigma_fun_small(N = N, G = G, idx = idx_by_cluster,
                        b = b, y = y_t, Z = Z,
                        Ainv = Ainv, sigma2 = sigma2, fhat = fhat, w = w)

    # (2.ii) Update D (random-effects covariance)
    D <- D_fun_small(G = G, b = b, Ainv = Ainv)

    # --- Inner-loop convergence check (GLL stabilization) ---
    gll[m + 2] <- gll_fun(D = D, b = b, idx = idx_by_cluster,
                          Z = Z, y = y_t, fhat = fhat, s2 = sigma2, w = w)
    if (m > 1L) {
      rel <- abs(gll[m + 2] - gll[m + 1]) / (abs(gll[m + 1]) + 1e-12)
      if (rel < tol) { n_iter <- m; converged_in_t <- TRUE; break }
    }

    if (m >= max_iter_inn) {                  # max-iteration guard
      n_iter <- m
      converged_in_t <- FALSE
      # message(sprintf("WARNING: the EM algorithm did not converge in %d iterations.", max_iter_inn)
      break
    }
  }

  # --- Outer-loop update (PQL step) ---
  converged_in <- c(converged_in, converged_in_t)
  zb <- numeric(N)
  for (g in seq_len(G)) {
    idx <- idx_by_cluster[[g]]
    zb[idx] <- Z[idx, , drop = FALSE] %*% b[g, ]
  }
  eta <- fhat + zb                            # recompute linear predictor

  # (Outer stopping rule - paper style)
  d_eta <- sqrt(mean((eta - eta_old)^2))      # RMS change of eta
  if (d_eta < tol) {
```

```r
      converged_out <- TRUE
      break
    }

    # Update working quantities for next outer iteration
    eta_old <- eta
    mu <- exp(eta) / (1 + exp(eta))             # updated conditional means
    y_t <- log(mu / (1 - mu)) + (y - mu) / (mu * (1 - mu))  # new pseudo-response
    w <- mu * (1 - mu)                          # new working weights

    M <- M + 1L
    if (M >= max_iter_out) {                    # guard against outer non-convergence
      converged_out <- FALSE
      message(sprintf("WARNING: the PQL algorithm did not converge in %d iterations.", max_iter_out))
      break
    }
  }

  # --- Stop timer and compute elapsed time ---
  time_end <- proc.time()
  elapsed <- as.numeric((time_end - time_start)["elapsed"])   # total runtime in seconds


  # --- Return fitted components ---
  list(
    tree = tree,                    # fitted rpart tree (fixed-effects function f(X))
    b = b,                          # estimated random effects (G x q)
    D = D,                          # estimated random-effects covariance
    sigma2 = sigma2,                # estimated residual variance
    mu = mu,                        # conditional means
    converged_in = converged_in,    # convergence flags for inner loops
    converged_out = converged_out,  # convergence flag for outer loop
    n_iter = n_iter,                # iterations performed (inner loop)
    train_ids = df$id,              # cluster identifiers in training set
    gll = gll,                      # GLL trace (for diagnostics)
    tol = tol,                      # convergence tolerance used
    time = elapsed                  # total runtime (seconds)
  )
}
```

Obviously we can do the same for the GMERF model by replacing the relevant functions with their small-system versions.

```r
fit_gmerf_small    <- function(df,                 # df: data.frame with columns
                                            #   id   : cluster identifier (factor or integer)
                                            #   y    : numeric response
                                            #   x1,x2,x3 : numeric covariates
                                            #   leaf : true generating leaf (optional, for diagnostics)
                          max_iter_inn = 1000,  # maximum number of EM iterations (inner loop)
                          max_iter_out = 1000,  # maximum number of PQL iterations (outer loop)
                          tol = 1e-6,           # convergence tolerance for both loops (Aitken or relati
                          ntrees = 500,        # RF: number of trees
                          mtry   = NULL,       # RF: features tried at each split (default = floor(p/3)
```

```r
                        min_node_size = 5,    # RF: terminal node size
                        max.depth = NULL,     # RF: optional max depth (NULL = unlimited)
                        seed = 1234           # random seed for reproducibility
                        ) {

# --- Basic setup ---
N <- nrow(df)                                 # total number of observations
G <- length(unique(df$id))                    # number of clusters
idx_by_cluster <- split(seq_len(N), df$id)    # list: row indices grouped by cluster

y <- df$y                                     # response vector
Z <- cbind(1, df$x1)                          # random-effects design: intercept + slope on x1
q <- ncol(Z)                                  # number of random effects (q = 2)

# --- Initialization (Step 0) ---
M <- 0L                                       # outer-loop counter
mu <- ifelse(y == 1, 0.75, 0.25)              # initial conditional means
y_t <- log(mu / (1 - mu)) + (y - mu) / (mu * (1 - mu))  # initial pseudo-response (PQL linearization)
w <- mu * (1 - mu)                            # initial working weights
sigma2 <- 1                                   # initial residual variance
D <- diag(2)                                  # initial random-effects covariance (identity)
b <- matrix(0, G, q)                          # initialize cluster random effects
gll <- c(0, 0)                                # GLL storage (2 slots for Aitken acceleration)
eta_old <- rep(0, N)                          # previous eta for outer-loop convergence check
converged_in <- c()                           # inner-loop convergence flags (per outer iteration)
converged_out <- FALSE                        # outer-loop convergence flag

# --- Outer loop (PQL updates) ---
repeat {
  m = 0                                       # reset inner-loop counter

  # --- Inner loop (EM-like iterations) ---
  repeat {
    m <- m + 1L

    # (1.i) Partial E-step: compute adjusted pseudo-response y_tilde* = y_tilde - Z b
    zb <- numeric(N)                          # cluster-specific random contributions
    for (g in seq_len(G)) {
      idx <- idx_by_cluster[[g]]              # indices for cluster g
      zb[idx] <- Z[idx, , drop = FALSE] %*% b[g, ]  # Z_i b_i
    }
    y_star <- y_t - zb                        # adjusted response for tree fit

    # (1.ii) M-step: fit random forest to (y_tilde*, X, W)
    Xrf <- data.frame(x1 = df$x1, x2 = df$x2, x3 = df$x3)
    rf <- ranger::ranger(
      formula         = y_star ~ .,
      data            = cbind(y_star = y_star, Xrf),
      case.weights    = w,
      num.trees       = ntrees,
      min.node.size   = min_node_size,
      max.depth       = max.depth,
      respect.unordered.factors = TRUE,
```

```r
      importance      = "none",
      write.forest    = TRUE,
      seed            = seed
    )
    fhat <- as.numeric(predict(rf, data = Xrf)$predictions)

    # (1.iii) Update random effects b_i
    Ainv <- Ajnv_fun(Z = Z, D = D, sigma2 = sigma2, G = G, idx = idx_by_cluster, w = w)  # build V_i
    b <- b_fun_small(G = G, Z = Z, idx = idx_by_cluster,
               y_t = y_t, fhat = fhat, w = w, sigma2 = sigma2, Ainv = Ainv)

    # (2.i) Update sigma^2 (residual variance)
    sigma2 <- sigma_fun_small(N = N, G = G, idx = idx_by_cluster,
                        b = b, y = y_t, Z = Z,
                        Ainv = Ainv, sigma2 = sigma2, fhat = fhat, w = w)

    # (2.ii) Update D (random-effects covariance)
    D <- D_fun_small(G = G, b = b, Ainv = Ainv)

    # --- Inner-loop convergence check (GLL stabilization) ---
    gll[m + 2] <- gll_fun(D = D, b = b, idx = idx_by_cluster,
                          Z = Z, y = y_t, fhat = fhat, s2 = sigma2, w = w)
    if (m > 1L) {
      rel <- abs(gll[m + 2] - gll[m + 1]) / (abs(gll[m + 1]) + 1e-12)
      if (rel < tol) { n_iter <- m; converged_in_t <- TRUE; break }
    }

    if (m >= max_iter_inn) {                  # max-iteration guard
      n_iter <- m
      converged_in_t <- FALSE
      # message(sprintf("WARNING: the EM algorithm did not converge in %d iterations.", max_iter_inn)
      break
    }
  }

  # --- Outer-loop update (PQL step) ---
  converged_in <- c(converged_in, converged_in_t)
  zb <- numeric(N)
  for (g in seq_len(G)) {
    idx <- idx_by_cluster[[g]]
    zb[idx] <- Z[idx, , drop = FALSE] %*% b[g, ]
  }
  eta <- fhat + zb                            # recompute linear predictor

  # (Outer stopping rule - paper style)
  d_eta <- sqrt(mean((eta - eta_old)^2))      # RMS change of eta
  if (d_eta < tol) {
    converged_out <- TRUE
    break
  }

  # Update working quantities for next outer iteration
  eta_old <- eta
```

```r
    mu <- exp(eta) / (1 + exp(eta))             # updated conditional means
    y_t <- log(mu / (1 - mu)) + (y - mu) / (mu * (1 - mu))  # new pseudo-response
    w <- mu * (1 - mu)                          # new working weights

    M <- M + 1L
    if (M >= max_iter_out) {                    # guard against outer non-convergence
      converged_out <- FALSE
      message(sprintf("WARNING: the PQL algorithm did not converge in %d iterations.", max_iter_out))
      break
    }
  }

  # --- Return fitted components ---
  list(
    forest = rf$forest,           # fitted regression forest
    b = b,                         # estimated random effects (G x q)
    D = D,                         # estimated random-effects covariance
    sigma2 = sigma2,               # estimated residual variance
    mu = mu,                       # conditional means
    converged_in = converged_in,   # convergence flags for inner loops
    converged_out = converged_out, # convergence flag for outer loop
    n_iter = n_iter,               # iterations performed (inner loop)
    train_ids = df$id,             # cluster identifiers in training set
    gll = gll,                     # GLL trace (for diagnostics)
    tol = tol                      # convergence tolerance used
  )
}
```

Testing

```r
#  Simulate clustered binary data (30 clusters, 60 observations each)
df <- sim_data_gmert(G = 30, n_i = 60)

#  Fit the GMERT model using the full dataset
fit1 <- fit_gmert(df, max_iter_inn = 1000, max_iter_out = 100, tol = 1e-6)

fit2 <- fit_gmert_small(df, max_iter_inn = 1000, max_iter_out = 100, tol = 1e-6)

#  Compare results
round(fit1$b,10) == round(fit2$b,10)

fit1$D
fit2$D

fit1$sigma2
fit2$sigma2

# compare computation times
fit1$time
fit2$time
```